

Projeto em Nível RTL

Nome dos Integrantes: Felipe Cabello Russo Magalhães Silva e Felipe Santos Souza

Turma: PN1 Professor: Marconi de Oliveira Júnior

Projeto RTL - Elevador Inteligente

Introdução

Este trabalho marca o início do projeto final da disciplina de Sistemas Digitais, desenvolvido ao longo das aulas 11 a 15. O propósito desta etapa é escolher e estruturar um projeto em nível de transferência entre registradores (RTL), que será posteriormente implementado em FPGA. Nesta fase inicial, buscamos compreender o problema proposto, organizar as ideias e definir como o sistema será dividido entre caminho de dados e unidade de controle.

Durante a elaboração do projeto, discutimos quais sinais de entrada e saída seriam necessários, quais componentes digitais fariam parte da solução (como registradores, comparadores e lógica combinacional) e como a máquina de estados finitos (FSM) será responsável por coordenar o funcionamento do circuito. Também levamos em consideração os recursos disponíveis na placa FPGA, como botões, LEDs, switches, display de sete segmentos e o clock, que serão utilizados durante a implementação física.

Esta primeira entrega tem um papel fundamental: estabelecer uma base sólida para o desenvolvimento do sistema. Ela inclui a definição do tema, a motivação para sua escolha, a identificação de requisitos e a estrutura geral do controlador, garantindo que o projeto seja organizado, funcional e viável dentro do escopo da disciplina.

Resumo

Este documento apresenta a proposta inicial do projeto final da disciplina de Laboratório de Sistemas Digitais, cujo objetivo é desenvolver um sistema digital em nível RTL utilizando uma FPGA. O tema escolhido para implementação é um controlador inteligente de elevador, capaz de atender chamadas de dois andares, monitorar o estado das portas e reagir a situações de emergência.

O projeto conta com uma máquina de estados com seis etapas principais: inicialização, espera por comandos, movimentação, chegada ao andar desejado, emergência e reset. A lógica do sistema garante que o elevador só se mova com as portas devidamente fechadas e que situações críticas, como o acionamento do botão de emergência, interrompam o funcionamento imediatamente.

Além da FSM, serão utilizados registradores para armazenar informações como posição atual do elevador e estado das portas, além de comparadores e lógica combinacional para tomada de decisões. O uso da FPGA permitirá testar o comportamento do sistema na prática, utilizando botões, LEDs e outros periféricos para simulação do ambiente real.

Com este projeto, buscamos consolidar conhecimentos sobre lógica combinacional e sequencial, FSMs e implementação de circuitos digitais, aplicando os conceitos estudados em um exemplo prático e próximo de aplicações reais em automação.

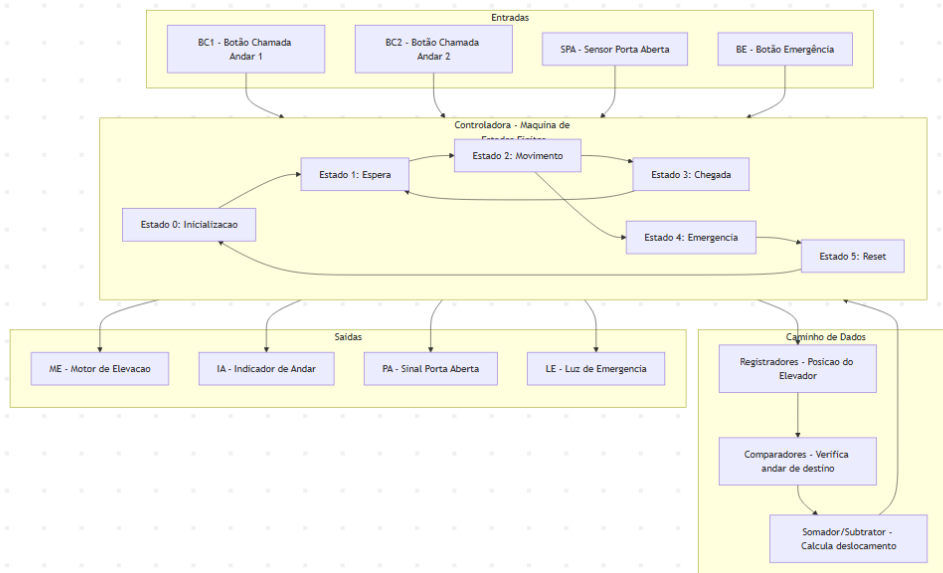


Figura 1: Diagrama de blocos do controlador de elevador inteligente

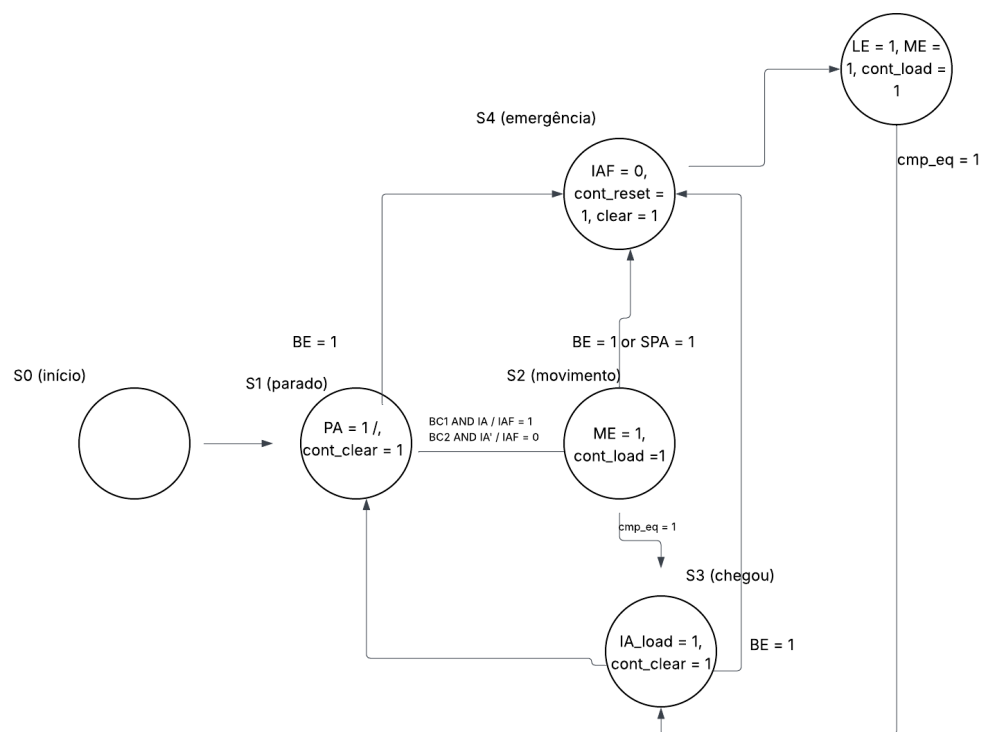


Figura 2: FSM da controladora do elevador inteligente

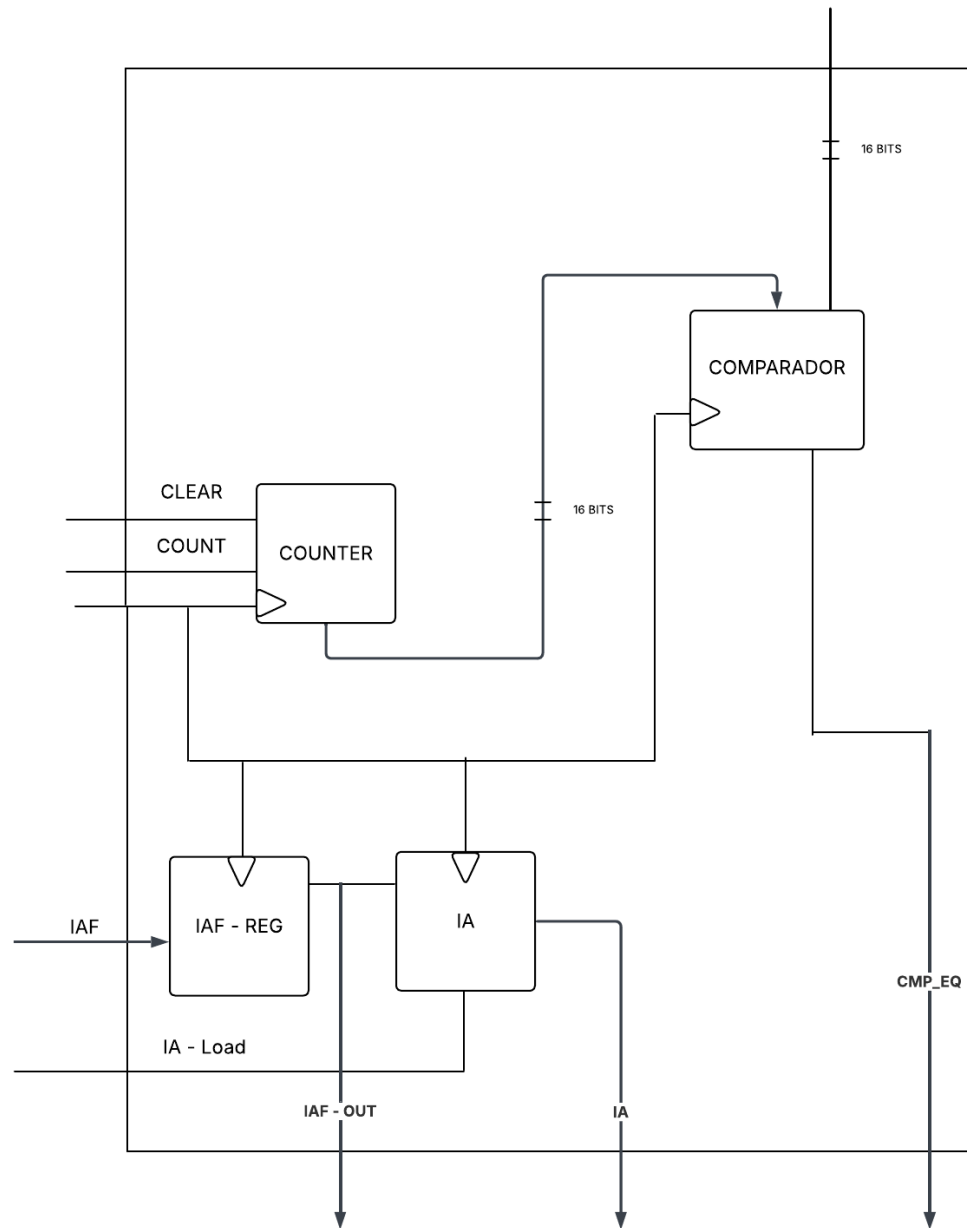


Figura 3: Datapath do elevador inteligente

1 Conclusão

O elevador inteligente de dois andares foi bem construído e executado, com um tempo de troca de andares de aproximadamente 4 segundos (visto que os elevadores normais demoram 2-6 segundos para uma troca de andar). Logo, o projeto final foi um sucesso.

Palavras-chave

Projeto RTL; FSM; FPGA; Circuitos Digitais; Caminho de Dados

Referências

[1] LUBASZEWSKI, M. S.; BRANQUINHO, L. C.; JOHANN, S. *VHDL Caipira: Uma Introdução Descomplicada ao Projeto Digital*. UFRGS, 2004.

2 Apêndice de códigos

2.1 Códigos EDA Playground

2.1.1 Sistema

```
library IEEE;
use IEEE.std_logic_1164.all;
use ieee.numeric_std.all;

entity sistema is
port(
    BC1, BC2, SPA, BE : in std_logic := '0'; --associar SPA a uma alavanca e PA a SPA
    CLOCK : in std_logic;
    PA, LE, ME, IA_saida, IAF_saida : out std_logic := '0'
);

end entity sistema;

architecture arch_sistema of sistema is

    signal cont_load, cont_clear, IA_load, IAF_carregado: std_logic := '0';
    signal IA, IAF : std_logic := '0';
    signal cmp_eq : std_logic := '0';

    component datapath is
    generic(W : natural := 16
    );
    port(
        cont_load, cont_clear : in std_logic;
        IAF : in std_logic;
        CLOCK : in std_logic;
        IA_load : in std_logic;
        IA : out std_logic;
        cmp_eq : out std_logic;
```

```

        Ciclos_comparador : std_logic_vector (15 downto 0);
        IAF_carregado : out std_logic
    );
end component;

component controladora is
port (
    BC1, BC2 : in std_logic;
    SPA : in std_logic;
    BE : in std_logic := '0';
    cmp_eq : in std_logic;
    IA : in std_logic := '0';
    CLOCK : in std_logic;
    ME : out std_logic;
    LE : out std_logic;
    PA : out std_logic;
    cont_load, cont_clear : out std_logic;
    IA_load : out std_logic;
    IAF : out std_logic
);
end component;

begin
    datapath_inst : datapath port map(
        cont_load => cont_load,
        cont_clear => cont_clear,
        IAF =>IAF,
        CLOCK => CLOCK,
        IA_load => IA_load,
        IA => IA,
        cmp_eq => cmp_eq,
        Ciclos_comparador => std_logic_vector(to_unsigned(150, 16)),
        IAF_carregado => IAF_carregado);

    controladora_inst : controladora port map(
        BC1 => BC1,
        BC2 => BC2,
        SPA => SPA,
        BE => BE,
        cmp_eq => cmp_eq,
        IA => IA,
        CLOCK =>CLOCK,
        ME => ME,
        LE => LE,
        PA => PA,

```

```

    cont_load => cont_load,
    cont_clear => cont_clear,
    IA_load => IA_load,
    IAF => IAF);

```

```

end architecture;

```

2.1.2 Testbench do Sistema

```

-- Code your testbench here
library IEEE;
use IEEE.std_logic_1164.all;

entity tb_sistema is
end tb_sistema;

architecture arch_tb_sistema of tb_sistema is
    component sistema is
    port(
        BC1, BC2, SPA, BE : in std_logic;
        CLOCK : in std_logic;
        PA, LE, ME, IA_saida, IAF_saida : out std_logic := '0'
    );

    end component;

    signal BC1, BC2, SPA, BE, CLOCK : std_logic := '0';
    signal PA, LE, ME, IA_saida, IAF_saida : std_logic := '0';
    begin

        DUT : sistema
        port map(
            BC1 => BC1,
            BC2 => BC2,
            SPA => SPA,
            BE => BE,
            CLOCK => CLOCK,
            PA => PA,
            LE => LE,
            ME => ME,
            IA_saida => IA_saida,
            IAF_saida => IAF_saida
        );
    end process

```

```

begin
  BC1  <= '0';
  BC2 <= '1';
    SPA <= '0';
    BE  <= '0';

  wait for 20 ns;
  CLOCK <= '1';
  wait for 20 ns;
  CLOCK <= '0';
  wait for 20 ns;
  CLOCK <= '1';
  wait for 20 ns;
  CLOCK <= '0';
  wait for 20 ns;
  CLOCK <= '1';
  wait for 20 ns;
  CLOCK <= '0';
  wait for 20 ns;
  CLOCK <= '1';
  wait for 20 ns;
  CLOCK <= '0';

  for i in 1 to 80 loop

    CLOCK <= '1';
    wait for 20 ns;
    CLOCK <= '0';
    wait for 20 ns;

    CLOCK <= '1';
    wait for 20 ns;
    CLOCK <= '0';
    wait for 20 ns;
  end loop;

  BE <= '1';

  CLOCK <= '1';
  wait for 20 ns;
  CLOCK <= '0';
  wait for 20 ns;

  CLOCK <= '1';
  wait for 20 ns;

```

```
CLOCK <= '0';  
wait for 20 ns;
```

```
CLOCK <= '1';  
wait for 20 ns;  
CLOCK <= '0';  
wait for 20 ns;
```

```
CLOCK <= '1';  
wait for 20 ns;  
CLOCK <= '0';  
wait for 20 ns;
```

```
for i in 1 to 80 loop
```

```
    CLOCK <= '1';  
    wait for 20 ns;  
    CLOCK <= '0';  
    wait for 20 ns;
```

```
    CLOCK <= '1';  
    wait for 20 ns;  
    CLOCK <= '0';  
    wait for 20 ns;
```

```
end loop;
```

```
BE <= '0';  
BC2 <= '0';
```

```
for i in 1 to 80 loop
```

```
    CLOCK <= '1';  
    wait for 20 ns;  
    CLOCK <= '0';  
    wait for 20 ns;
```

```
    CLOCK <= '1';  
    wait for 20 ns;  
    CLOCK <= '0';  
    wait for 20 ns;
```

```
end loop;
```

```

        wait;
    end process;

```

```

end architecture;

```

2.2 Datapath

```

library IEEE;
use IEEE.std_logic_1164.all;

entity datapath is
    generic(W : natural := 16
    );
    port(
        cont_load, cont_clear : in std_logic;
        IAF : in std_logic;
        CLOCK : in std_logic;
        IA_load : in std_logic;
        IA : out std_logic;
        cmp_eq : out std_logic;
        Ciclos_comparador : std_logic_vector (15 downto 0);
        IAF_carregado : out std_logic
    );
end entity datapath;

architecture arch_datapath of datapath is

    signal resultado_contador : std_logic_vector (15 downto 0);
    signal saida_IAF_reg : std_logic;
    signal cmp_saida : std_logic;

    component Contador is
        generic (
            W : natural := 16
        );
        port(
            CLOCK : in std_logic;
            RESET : in std_logic;

```

```

        CONT_LOAD : in std_logic;
        FINAL_VALUE : out std_logic_vector (W-1 downto 0)
    );
end component;

```

```

component Comparador_16 is
    Port (
        A : in std_logic_vector (15 downto 0) := (others => '0');
        B : in std_logic_vector (15 downto 0) := (others => '0');
        CLOCK : in std_logic;
        igual_ou_maior : out std_logic
    );
end component;

```

```

component reg_iaf is
    port (
        entrada_iaf : in std_logic := '0';
        carregar_bit_iaf : in std_logic := '1';
        CLOCK_iaf : in std_logic := '0';
        saida_iaf : out std_logic
    );
end component;

```

```

component reg_ia is
    port (
        entrada_ia : in std_logic := '0';
        carregar_bit_ia : in std_logic := '0';
        CLOCK_ia : in std_logic := '0';
        saida_ia : out std_logic
    );
end component;

```

```

component comp_reg is
    port (
        entrada_comp : in std_logic := '0';
        carregar_bit_comp : in std_logic := '1';
        CLOCK_comp : in std_logic := '0';
        saida_comp : out std_logic
    );
end component;

```

```

begin
    cont2: contador port map(CLOCK => CLOCK, RESET => cont_clear, CONT_LOAD => cont_lo

```

```

    comp: Comparador_16 port map(A => resultado_contador, B => Ciclos_comparador, CLOCK => CLOCK,
    reg_de_iaf: reg_iaf port map(entrada_iaf => IAF, carregar_bit_iaf => '1', CLOCK_iaf => CLOCK,
    reg_de_ia: reg_ia port map(entrada_ia => saida_IAF_reg, carregar_bit_ia => IA_load, CLOCK_ia => CLOCK,
    reg_do_comparador: comp_reg port map(entrada_comp => cmp_saida, carregar_bit_comp => cmp_eq, CLOCK_comp => CLOCK,
    IAF_carregado <= saida_IAF_reg;

end architecture;

```

2.2.1 Testbench do Datapath

```

-- Code your testbench here
library IEEE;
use IEEE.std_logic_1164.all;

entity tb_datapath is
end tb_datapath;

architecture tb_arch_datapath of tb_datapath is

    component datapath is
        generic(W : natural := 16)
        port(
            cont_load, cont_clear : in std_logic;
            IAF : in std_logic;
            CLOCK : in std_logic;
            IA_load : in std_logic;
            IA : out std_logic;
            cmp_eq : out std_logic;
            Ciclos_comparador : std_logic_vector (15 downto 0);
            IAF_carregado : out std_logic
        );
    end component;

    signal cont_load, cont_clear, IAF, CLOCK, IA_load, IA, cmp_eq, IAF_carregado : std_logic;
    signal Ciclos_comparador : std_logic_vector (15 downto 0);

begin

```

```
DUT : datapath port map (cont_load => cont_load, cont_clear => cont_clear, IAF => IA
```

```
process
begin
--testando contador (check)
cont_load <= '1';
cont_clear <= '0';

    --testando comparador (confirmar se o atraso tem algum efeito ou não, mas ok)
    Ciclos_comparador <= "0000000000000100";

--testando registradores (check)
IAF <= '1';
IA_load <= '1';
for i in 1 to 5 loop

    CLOCK <= '1';
    wait for 20 ns;
    CLOCK <= '0';
    wait for 20 ns;

    CLOCK <= '1';
    wait for 20 ns;
    CLOCK <= '0';
    wait for 20 ns;
end loop;

cont_clear <= '1';
cont_load <= '0';
IAF <= '0';
for i in 1 to 5 loop

    CLOCK <= '1';
    wait for 20 ns;
    CLOCK <= '0';
    wait for 20 ns;

    CLOCK <= '1';
    wait for 20 ns;
    CLOCK <= '0';
    wait for 20 ns;
end loop;
--testando registradores
    IA_load <= '0';
    IAF <= '1';
```

```

        CLOCK <= '1';
        wait for 20 ns;
        CLOCK <= '0';
        wait for 20 ns;

wait;
end process;

```

```

end architecture;

```

2.3 Controladora

2.3.1 Descrição

```

        library IEEE;
        use IEEE.std_logic_1164.all;

entity controladora is
port (
    BC1, BC2 : in std_logic;
    SPA : in std_logic;
    BE : in std_logic := '0';
    cmp_eq : in std_logic;
    IA : in std_logic := '0';
    CLOCK : in std_logic;
    ME : out std_logic;
    LE : out std_logic;
    PA : out std_logic;
    cont_load, cont_clear : out std_logic;
    IA_load : out std_logic;
    IAF : out std_logic
);
end entity controladora;

architecture arch_controladora of controladora is

    signal estado_atual, estado_futuro : std_logic_vector (2 downto 0) := "000";

    constant INI : std_logic_vector (2 downto 0) := "000"; --INI = estado inicial
    constant PAR : std_logic_vector (2 downto 0) := "001"; --PAR = parado
    constant MOV : std_logic_vector (2 downto 0) := "010"; --MOV = em movimento
    constant CHE : std_logic_vector (2 downto 0) := "011"; --CHE = chegou

```

```

constant EME : std_logic_vector (2 downto 0) := "100"; --EME = emergência
constant RES : std_logic_vector (2 downto 0) := "101"; --RES = reset

begin
  P1 : process(CLOCK)
  begin
    if (rising_edge(CLOCK)) then
      estado_atual <= estado_futuro;
    end if;
  end process P1;

  P2 : process(estado_atual,BE, BC1, BC2, SPA, cmp_eq, IA)
  begin
    if ((BE = '1') AND (estado_atual = CHE)) then
      cont_clear <= '1'; --talvez fazer 1 ciclo de clock extra ou eliminar o registrador
      IAF <= '0';
      LE <= '1';
      estado_futuro <= EME;
    elsif ((SPA = '1') AND (estado_atual = MOV)) then
      cont_clear <= '1';
      IAF <= '0';
      LE <= '1';
      estado_futuro <= EME;
    else
      case estado_atual is
        when INI =>
          estado_futuro <= "001";
          PA <= '1';
          cont_load <= '0';
          cont_clear <= '1';
          IA_load <= '0';
          LE <= '0';
          ME <= '0';

          when PAR =>

            PA <= '1';
            cont_load <= '0';
            cont_clear <= '1';
            --talvez adicionar um clear no reg do comparador
            IA_load <= '0';
            LE <= '0';
            ME <= '0';

```

```

if (BC1 = '1' AND IA = '1') then
    estado_futuro <= MOV;
    IAF <= '0';
elseif (BC2 = '1' AND IA = '0') then
    estado_futuro <= MOV;
    IAF <= '1';
end if;

when MOV =>

    ME <= '1';
    PA <= '0';
    cont_clear <= '0';
    cont_load <= '1';
    IA_load <= '0';
    --IAF mantém
    LE <= '0';
    if (cmp_eq = '1') then
        estado_futuro <= CHE;
    end if;

when CHE =>
    IA_load <= '1'; --como eu faço IA = 1??
    cont_load <= '0';
    cont_clear <= '1';
    --IAF mantém
    LE <= '0';
    ME <= '0';
    PA <= '0';
    estado_futuro <= PAR;

when EME => --fazer os outputs do EME e RES
    cont_load <= '0';
    LE <= '1';
    ME <= '0';
    PA <= '0';
    IA_load <= '0';
    IAF <= '0';
    cont_clear <= '1';
    estado_futuro <= RES;

when RES =>
    cont_clear <= '0';
    --IAF mantém
    ME <= '1';

```

```

        PA <= '0';
        IA_load <= '0';
        cont_load <= '1';
        LE <= '1';
        if (cmp_eq = '1') then
            estado_futuro <= CHE;
        end if;

    when others =>
        cont_load <= '0';
        cont_clear <= '0';
        IA_load <= '0';
        IAF <= '0';
        LE <= '0';
        ME <= '0';
        PA <= '0';
    end case;
end if;
end process P2;
end arch_controladora;

```

2.3.2 Testbench da Controladora

```

-- Code your testbench here
library IEEE;
use IEEE.std_logic_1164.all;

entity tb_controladora is
end entity tb_controladora;

architecture arch_tb_controladora of tb_controladora is
    component controladora is
    port (
        BC1, BC2 : in std_logic;
        SPA : in std_logic;
        BE : in std_logic;
        cmp_eq : in std_logic;
        IA : in std_logic;
        CLOCK : in std_logic;
        ME : out std_logic;
        LE : out std_logic;
    );
    end component;

```

```

    PA : out std_logic;
    cont_load, cont_clear : out std_logic;
    IA_load : out std_logic;
    IAF : out std_logic
);
end component;

signal BC1, BC2, SPA, BE, cmp_eq, CLOCK, PA, cont_load, cont_clear, IA_load, IAF, LE,
signal IA : std_logic := '0';

begin
    DUT: controladora port map (BC1 => BC1, BC2 => BC2, SPA => SPA, BE => BE, cmp_eq => c

    process
    begin
        ---estou no estado 1 - quero passar para o "parado"
        CLOCK <= '1';

        --agora quero ir para MOV
        wait for 20 ns;
        CLOCK <= '0';
        BC2 <= '1';
        IA <= '0';

        wait for 20 ns;
        --ok, deu certo, agora para CHE (simulando um tempo para chegar em cmp_eq

        for i in 1 to 4 loop
            CLOCK <= '1';
            wait for 20 ns;
            CLOCK <= '0';
            wait for 20 ns;

        end loop;
        BC2 <= '1';
        cmp_eq <= '1';
        --se estivesse ligado com o datapath, IA_load faria IA = '1', depois de 1 ciclo de clo
        CLOCK <= '1';
        wait for 20 ns;
        CLOCK <= '0';
        wait for 20 ns;
        IA <= '1';

        for i in 1 to 4 loop
            CLOCK <= '1';

```

```

        wait for 20 ns;
        CLOCK <= '0';
        wait for 20 ns;
    end loop;
--agora para o botão de emergência
    BC1 <= '1';
    cmp_eq <= '0';

    CLOCK <= '1';
    wait for 20 ns;
    CLOCK <= '0';
    wait for 20 ns;

    BE <= '1';
    for i in 1 to 4 loop
        CLOCK <= '1';
        wait for 20 ns;
        CLOCK <= '0';
        wait for 20 ns;
    end loop;

    cmp_eq <= '1';
    CLOCK <= '1';
    wait for 20 ns;
    CLOCK <= '0';
    wait for 20 ns;

    CLOCK <= '1';
    wait for 20 ns;
    CLOCK <= '0';
    wait for 20 ns;

    wait;
end process;

end arch_tb_controladora;

```

2.4 Contador

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity Contador is
    generic (

```

```

        W : natural := 16
    );
    port(
        CLOCK : in std_logic;
        RESET : in std_logic;
        CONT_LOAD : in std_logic;
        FINAL_VALUE : out std_logic_vector (W-1 downto 0)
    );
end entity Contador;

architecture Main of Contador is
    signal COUNT : unsigned(W-1 downto 0) := (others => '0');
begin
    process (CLOCK, RESET)
    begin
        if RESET = '1' then
            COUNT <= (others => '0');
        elsif rising_edge(CLOCK) then
            if (CONT_LOAD = '1') then
                COUNT <= COUNT + 1;
            end if;
        end if;
    end process;

    FINAL_VALUE <= std_logic_vector(COUNT);
end architecture Main;

```

2.4.1 Testbench do Contador

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity tb_contador is
end tb_contador;

architecture teste of tb_contador is
    component contador is
        generic (
            W : natural := 16
        );
        port(
            CLOCK : in std_logic;
            RESET : in std_logic;
            CONT_LOAD : in std_logic;

```

```

    FINAL_VALUE : out std_logic_vector (W-1 downto 0)
);
end component;
constant period: time := 25 NS;

    signal CLOCK: std_logic;
    signal RESET,CONT_LOAD : std_logic;
    signal FINAL_VALUE : std_logic_vector(15 downto 0);

begin
    DUT: Contador port map (
    CLOCK => CLOCK, RESET => RESET, CONT_LOAD => CONT_LOAD, FINAL_VALUE => FINAL_VALUE);

processo: process
begin
    CONT_LOAD <= '1';
    RESET <= '0';
    for i in 1 to 10 loop
        CLOCK <= '1';
        wait for Period/2;
        CLOCK <= '0';
        wait for Period/2;
    end loop;

    RESET <= '1';
    CONT_LOAD <= '0';
    for i in 1 to 10 loop
        CLOCK <= '1';
        wait for Period/2;
        CLOCK <= '0';
        wait for Period/2;
    end loop;

    CONT_LOAD <= '1';
    RESET <= '0';
    for i in 1 to 10 loop
        CLOCK <= '1';
        wait for Period/2;
        CLOCK <= '0';
        wait for Period/2;
    end loop;

    wait;
end process processo;

```

```
end teste;
```

2.4.2 Registrador

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity reg is
  port (  entrada : in std_logic := '0';
         carregar_bit : in std_logic := '0';
         CLOCK : in std_logic := '0';
         saida : out std_logic
  );

end reg;

architecture arch_registrador of reg is
begin
  process (CLOCK)
  begin

    if rising_edge(CLOCK) then
      if carregar_bit = '1' then
        saida <= entrada;
      else
        end if;
    else
      end if;
    end process;
end arch_registrador;
```

2.4.3 Testbench do Registrador

```
library IEEE;
use IEEE.std_logic_1164.all;

entity tb_reg is
end tb_reg;

architecture arch_tb_registrador of tb_reg is

  component reg is
```

```

    port (entrada : in std_logic;
          carregar_bit : in std_logic;
          CLOCK : in std_logic;
          saida : out std_logic
    );
end component;

signal entrada : std_logic := '0';
signal carregar_bit : std_logic := '0';
signal CLOCK : std_logic := '0';
signal saida : std_logic := '0';

begin

    DUT: reg port map (
        entrada => entrada,
        saida => saida,
        carregar_bit => carregar_bit,
        CLOCK => CLOCK
    );

    process
    begin

        carregar_bit <= '1';
        entrada <= '1';
        CLOCK <= '1';
        wait for 100 ns;
        CLOCK <= '0';
        wait for 100 ns;

        carregar_bit <= '1';
        entrada <= '0';
        CLOCK <= '1';
        wait for 100 ns;
        CLOCK <= '0';
        wait for 100 ns;

        carregar_bit <= '1';
        entrada <= '1';
        CLOCK <= '1';
        wait for 100 ns;
        CLOCK <= '0';
        wait for 100 ns;
    end process;

```

```

        carregar_bit <= '0';
        entrada <= '0';
        CLOCK <= '1';
        wait for 100 ns;
        CLOCK <= '0';
        wait for 100 ns;

    wait;
    end process;

end arch_tb_registrador;

```

2.4.4 Comparador

```

    library IEEE;
    use IEEE.std_logic_1164.all;
    use IEEE.numeric_std.all;

    entity comparador_16 is

        Port (
            A : in std_logic_vector (15 downto 0) := (others => '0');
            B : in std_logic_vector (15 downto 0) := (others => '0');
            CLOCK : in std_logic;
            igual_ou_maior : out std_logic
        );

    end comparador_16;

    architecture arq_comparador of comparador_16 is
    begin

        process(CLOCK)
        begin

            if rising_edge(CLOCK) then
                if (unsigned(A) < unsigned(B)) then
                    igual_ou_maior <= '0';
                else
                    igual_ou_maior <= '1';
                end if;
            end if;
        end if;
    end process;
end arq_comparador;

```

```

    end process;
end arq_comparador;

```

2.4.5 Testbench do Comparador

```

library IEEE;
use IEEE.std_logic_1164.all;

entity tb_comparador_16 is
end tb_comparador_16;

architecture tb_arch_comparador_16 of tb_comparador_16 is

    component comparador_16 is
        Port (
            A : in std_logic_vector (15 downto 0);
            B : in std_logic_vector (15 downto 0);
            CLOCK : in std_logic;
            igual_ou_maior : out std_logic
        );
    end component;

    signal A, B : std_logic_vector (15 downto 0) := (others => '0');
    signal CLOCK : std_logic := '0';
    signal igual_ou_maior : std_logic;

begin
    DUT: comparador_16 port map (
        A => A,
        B => B,
        CLOCK => CLOCK,
        igual_ou_maior => igual_ou_maior
    );

    process
    begin

        A <= "0000000000000000";
        B <= "00000000100000000";
        CLOCK <= '1';
        wait for 10 ns;
        CLOCK <= '0';
        wait for 10 ns;
    end process;
end tb_arch_comparador_16;

```

```

A <= "00000001000000000";
B <= "00000000100000000";
CLOCK <= '1';
wait for 10 ns;
CLOCK <= '0';
wait for 10 ns;

A <= "00000000100000000";
B <= "00000000100000000";
CLOCK <= '1';
wait for 10 ns;
CLOCK <= '0';
wait for 10 ns;

A <= "00000000000000000";
B <= "00000000100000000";
CLOCK <= '1';
wait for 10 ns;
CLOCK <= '0';
wait for 10 ns;

wait;
end process;
end tb_arch_comparador_16;

```

2.5 Código para síntese

2.5.1 Elevador.vhd

```

library IEEE;
use IEEE.std_logic_1164.all;
use ieee.numeric_std.all;

entity Eleva_dor is
    port(
        SW : in std_logic_vector (9 downto 0);    -- SW0 = BC1; SW1 = BC2; SW2 = SPA; KEYC
        KEY : in std_logic_vector (1 downto 0);
        LEDR : out std_logic_vector (9 downto 0); --LEDRO = PA; LEDR1 = ME; LEDR2 = IA_sai
        CLOCK_50 : in std_logic
    );
end entity Eleva_dor;

architecture arch_sistema of Eleva_dor is

```

```

signal cont_load, cont_clear, IA_load, IAF_carregado : std_logic := '0';
signal IA, IAF : std_logic := '0';
signal cmp_eq : std_logic := '0';
signal CLOCK : std_logic;

component datapath is
    generic (
        W : natural := 16
    );
    port(
        cont_load, cont_clear : in std_logic;
        IAF : in std_logic;
        CLOCK : in std_logic;
        IA_load : in std_logic;
        IA : out std_logic;
        cmp_eq : out std_logic;
        Ciclos_comparador : std_logic_vector (15 downto 0);
        IAF_carregado : out std_logic
    );
end component;

component controladora is
    port (
        BC1, BC2 : in std_logic;
        SPA : in std_logic;
        BE : in std_logic := '0';
        cmp_eq : in std_logic;
        IA : in std_logic := '0';
        CLOCK : in std_logic;
        ME : out std_logic;
        LE : out std_logic;
        PA : out std_logic;
        cont_load, cont_clear : out std_logic;
        IA_load : out std_logic;
        IAF : out std_logic
    );
end component;

component DivisorClock is
    port (
        CLOCK_50MHz : in std_logic;
        reset       : in std_logic;
        CLOCK_4Hz   : out std_logic
    );

```

```

end component;

begin
  clock_inst: DivisorClock port map (
    CLOCK_50MHz => CLOCK_50,
    reset => '0',
    CLOCK_4Hz => CLOCK--mudar para 4 Hz
  );
  datapath_inst: datapath port map(
    cont_load => cont_load,
    cont_clear => cont_clear,
    IAF => IAF,
    CLOCK => CLOCK,
    IA_load => IA_load,
    IA => IA,
    cmp_eq => cmp_eq,
    Ciclos_comparador => std_logic_vector(to_unsigned(150, 16)),
    IAF_carregado => IAF_carregado
  );

  controladora_inst: controladora port map(
    BC1 => SW(0),
    BC2 => SW(1),
    SPA => SW(2),
    BE => KEY(0),
    cmp_eq => cmp_eq,
    IA => IA,
    CLOCK => CLOCK,
    ME => LEDR(1),
    LE => LEDR(9),
    PA => LEDR(0),
    cont_load => cont_load,
    cont_clear => cont_clear,
    IA_load => IA_load,
    IAF => IAF
  );

  --LEDR0 = PA; LEDR1 = ME; LEDR2 = IA_saida; LEDR_3 = IAF_saida; LEDR9 = LE;
  LEDR(2) <= IA;
  LEDR(3) <= IAF;
  LEDR(5) <= CLOCK;

end architecture;

```

2.5.2 Divisor_{clock}.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity DivisorClock is
  port
  (
    CLOCK_50MHz : in std_logic;
    reset       : in std_logic;
    CLOCK_4Hz   : out std_logic
  );

end entity;

architecture rtl of DivisorClock is

begin

  process (CLOCK_50MHz,reset)
    variable cnt : integer range 0 to 50000000;
    variable clk : std_logic:='0';
  begin
    if (reset = '1') then
      cnt := 0;
    elsif (rising_edge(CLOCK_50MHz)) then
      if (cnt = 650000) then
        clk := not clk;
        cnt := 0;
      else
        cnt := cnt + 1;
      end if;
    end if;
    CLOCK_4Hz <= clk;
  end process;

end rtl;
```